Entrega 2 – Engenharia de Software e Arquitetura de Sistemas

Aplicação de design de software e diagramas UML no PI

Projeto: Engenharia Software e Arquitetura de Sistemas

Professor orientador: Luiz

Unidade Curricular: Engenharia de Software e Arquitetura de Sistemas

Curso: Ciência da Computação – 4º semestre

Integrante	RA
Kaio Inglez	24026741
Mauricio Suster	24026985
Yuri Santana	25027408
Bruno nobrega	25027405

Design de Software

A plataforma foi pensada como uma sequência de etapas de dados em Python, e não como um sistema transacional. Ela lê o arquivo enviado pela CTI, prepara os dados, calcula os indicadores e mostra os resultados em um dashboard web. A seguir estão as principais decisões de design e o motivo de cada uma.

Arquitetura em camadas. Dividimos a solução em quatro camadas: dados brutos, preparação, dados processados e apresentação. Cada camada só usa o que a camada anterior entrega. Assim, uma mudança na tela não mexe no cálculo, e uma mudança no cálculo não exige mexer na leitura do arquivo.

Dado bruto preservado. O arquivo original da CTI (Demonstrativo Fecap v3.csv) nunca é alterado. Toda transformação gera um arquivo novo, então sempre dá para voltar ao dado de origem e refazer o processo do zero.

Arquivos em vez de banco de dados. A base tem 12 anos × 1.200 cenários, ou seja, 14.400 combinações, e cabe na memória de um computador comum. Por isso usamos arquivos CSV e XLSX, que são formatos abertos e fáceis de conferir. Um banco de dados daria mais trabalho de instalação sem trazer ganho para esse volume.

Cálculo em lote, separado da tela. Os cálculos rodam uma vez em scripts Python e geram um arquivo pronto para o dashboard. O dashboard só lê esse arquivo e aplica os filtros, por isso abre rápido. Também usamos o cache do Streamlit para não ler o arquivo de novo a cada clique.

Uma fonte única para as regras. Cada fórmula existe em um único script. O dashboard e a planilha de validação usam os mesmos resultados, o que evita ter duas versões da mesma conta com números diferentes.

Código modular. Cada arquivo tem uma função clara: preparar os dados, calcular, gerar o arquivo do dashboard, montar as telas e testar. Usamos funções pequenas, nomes em português e um comentário no início de cada arquivo explicando o que ele faz e como rodar.

Teste automático. Criamos o script testar_painel.py, que compara os valores mostrados no dashboard com a planilha de validação. Na última execução foram 864 comparações, todas iguais.

Tecnologias. Usamos Python com pandas para os dados, Google Colab para os notebooks, Streamlit e Plotly para o dashboard, e Git com GitHub para o versionamento. Escolhemos essa stack porque é a sugerida no projeto e porque o grupo já trabalha com Python.

Segurança e privacidade. Os dados da CTI são confidenciais, então o arquivo bruto não vai para repositório aberto. O repositório não guarda senhas nem chaves, e a publicação dos dados só acontece com autorização da CTI.

Usabilidade e tratamento de erros. O dashboard mostra os números no padrão brasileiro e com a unidade de cada valor. Quando falta um dado, aparece "n/a" com uma explicação. Se o arquivo de dados não existir, o sistema mostra uma mensagem dizendo qual script rodar antes.

Diagramas UML

Fizemos dois diagramas: o de casos de uso, que mostra quem usa a plataforma e para quê, e o de implantação com componentes, que mostra onde cada parte do sistema roda. Abaixo está a estrutura de cada um, com todos os elementos e ligações, para desenhar depois em uma ferramenta como draw.io ou Astah.

Diagrama de Casos de Uso

O sistema é um retângulo com o nome "Plataforma Analítica de Planejamento Financeiro". Os casos de uso ficam dentro dele, como elipses, e os atores ficam do lado de fora, como bonecos.

Ator	Tipo	Quem é
Equipe do projeto	Principal	Integrantes do grupo, que preparam os dados, rodam os cálculos e publicam o dashboard
Gestor da CTI	Principal	Quem consulta os indicadores no dashboard para apoiar decisões
Professor avaliador	Principal	Quem consulta o dashboard e reproduz os resultados pelo repositório
CTI Global	Secundário	Empresa que fornece o arquivo de dados

Código	Caso de uso	Ator	O que acontece
UC01	Receber arquivo de dados	Equipe do projeto, CTI Global	A CTI envia o DADOS_FECAP.zip e a equipe guarda o original sem alterar
UC02	Preparar dados	Equipe do projeto	Lê, limpa e padroniza o CSV e grava a base processada
UC03	Validar estrutura do arquivo	Incluído em UC02	Confere colunas, codificação e formato dos números antes de preparar
UC04	Calcular indicadores	Equipe do projeto	Roda os scripts de cálculo para os 1.200 cenários e 12 anos
UC05	Conferir com a planilha de validação	Incluído em UC04	Roda o teste que compara os resultados com a planilha
UC06	Gerar dados do dashboard	Equipe do projeto	Monta o arquivo dados_painel.csv que o dashboard lê
UC07	Publicar dashboard	Equipe do projeto	Envia o código para o GitHub e publica na nuvem
UC08	Consultar indicadores	Gestor da CTI, Professor avaliador	Abre o dashboard e vê cartões, gráficos e tabelas
UC09	Filtrar por cenário e ano	Incluído em UC08	Escolhe o cenário (1 a 1.200) e o ano (1 a 12)
UC10	Comparar com a mediana dos cenários	Estende UC08	Mostra nos gráficos a mediana dos 1.200 cenários

Código	Caso de uso	Ator	O que acontece
UC11	Exportar tabela em CSV	Estende UC08	Baixa a tabela que está na tela
UC12	Consultar notas metodológicas	Gestor da CTI, Professor avaliador	Vê fórmulas, conferências e limitações
UC13	Reproduzir resultados	Equipe do projeto, Professor avaliador	Clona o repositório, instala o requirements.txt e roda os scripts na ordem do README

Ligações do diagrama:

- **Associação** (linha contínua entre ator e caso de uso): Equipe do projeto com UC01, UC02, UC04, UC06, UC07 e UC13; Gestor da CTI com UC08 e UC12; Professor avaliador com UC08, UC12 e UC13; CTI Global com UC01.
- **«include»** (seta tracejada saindo do caso principal): UC02 para UC03, UC04 para UC05 e UC08 para UC09. O caso incluído sempre acontece junto.
- **«extend»** (seta tracejada saindo do caso opcional): UC10 para UC08 e UC11 para UC08. O usuário só faz se quiser.
- **Nota** (retângulo com canto dobrado ligado por linha tracejada a UC06): "Ordem de execução: UC02, depois UC04, depois UC06".

Para organizar o desenho, coloque Equipe do projeto e CTI Global à esquerda e Gestor da CTI e Professor avaliador à direita. Os casos de preparação e cálculo ficam na parte de cima e os de consulta na parte de baixo.

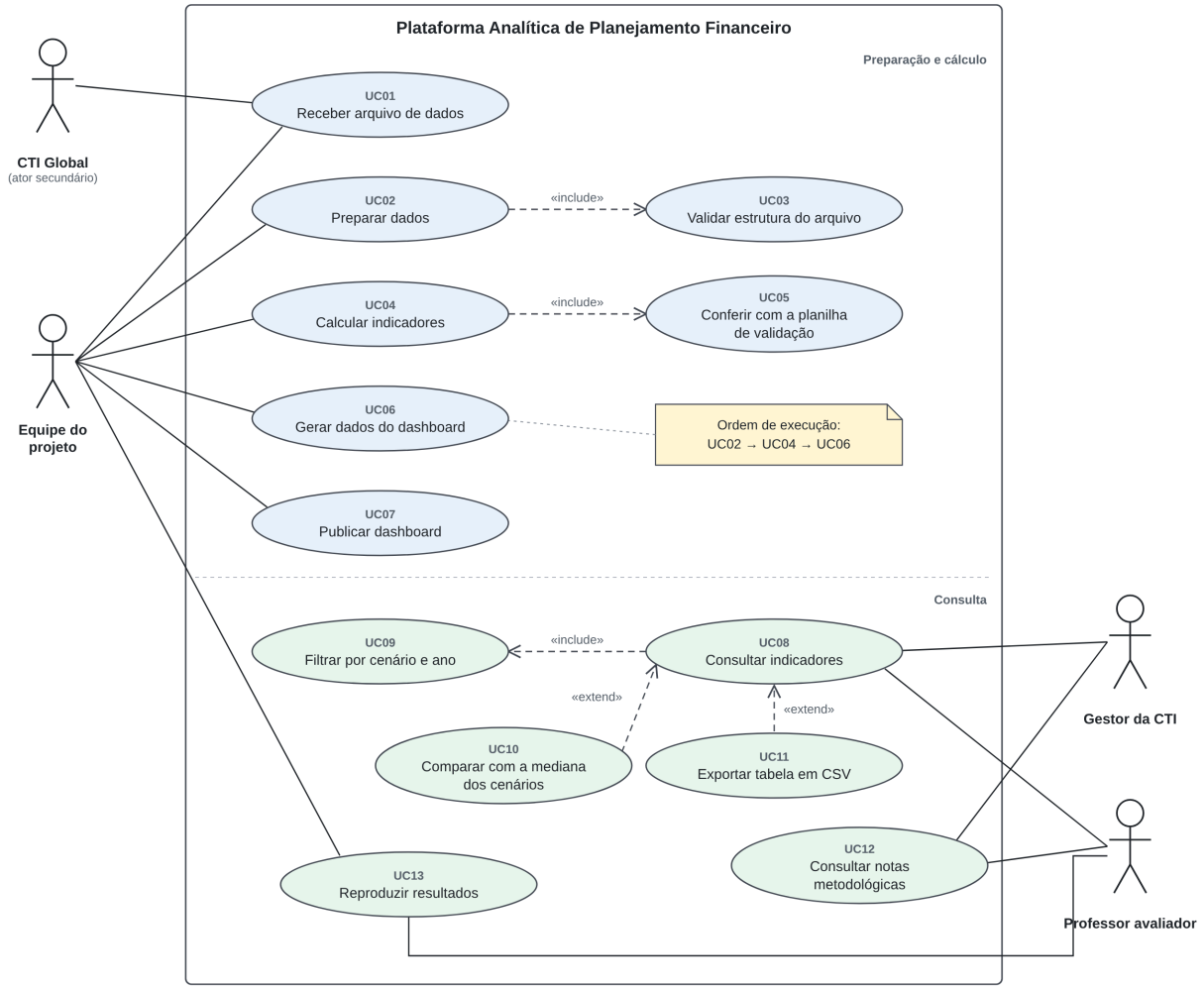


Figura 1 – Diagrama de Casos de Uso da plataforma.

Diagrama de Implantação e Componentes

Este diagrama junta os nós físicos (caixas em 3D) com os componentes de software que rodam em cada um e os arquivos que eles trocam.

Nó	Estereótipo	O que roda nele
Computador da equipe	«device» Windows com Python 3	Scripts de cálculo, teste e geração de dados; Git
Google Colab	«executionEnvironment»	Notebook de preparação dos dados
GitHub	«executionEnvironment»	Repositório com código, documentação e requirements.txt
Streamlit Community Cloud	«executionEnvironment»	Aplicação do dashboard

Nó	Estereótipo	O que roda nele
Navegador do usuário	«device»	Telas do dashboard

Componente	Arquivo	Nó	Responsabilidade	Depende de
Preparação de dados	Preparacao_Dados_CTI.ipynb	Google Colab	Lê o CSV bruto, limpa e grava base_cti_processada.csv	Arquivo bruto
Cálculo de KPIs	calculo_kpis.py	Computador da equipe	Calcula os KPIs e grava kpis_todos_cenarios.csv	Arquivo bruto
Cálculo de indicadores	calculo_entrega2.py	Computador da equipe	Calcula os indicadores e grava indicadores_e2_todos_cenarios.csv	Arquivo bruto
Geração dos dados do dashboard	preparar_dados.py	Computador da equipe	Seleciona as colunas e grava dados_painel.csv	Cálculo de indicadores
Teste de validação	testar_painel.py	Computador da equipe	Compara dados_painel.csv com a planilha de validação	dados_painel.csv, planilha de validação
Regras e tabelas	calculos.py	Streamlit Community Cloud	Lista os indicadores e monta as tabelas das telas	dados_painel.csv
Interface do dashboard	app.py	Streamlit Community Cloud	Telas, filtros, gráficos e mensagens de erro	Regras e tabelas, Streamlit, Plotly

Artefatos (retângulo com «artifact»), desenhados dentro do nó onde ficam:

- Demonstrativo Fecap v3.csv: dado bruto da CTI, fica só no computador da equipe e no Colab, fora de repositório aberto.
- base_cti_processada.csv, kpis_todos_cenarios.csv e indicadores_e2_todos_cenarios.csv: dados processados.
- dados_painel.csv: arquivo que o dashboard lê, com cerca de 10 MB.
- Planilha de validação (.xlsx) e requirements.txt.

Ligações entre os nós (linhas contínuas com o protocolo escrito em cima):

- Computador da equipe e GitHub: git push e git pull, por HTTPS.
- Computador da equipe e Google Colab: envio do DADOS_FECAP.zip pelo navegador.
- GitHub e Streamlit Community Cloud: publicação a cada push no ramo principal.
- Navegador do usuário e Streamlit Community Cloud: acesso ao dashboard por HTTPS.

As dependências entre componentes são setas tracejadas, seguindo a coluna "Depende de" da tabela. A seta sempre aponta para o componente ou arquivo que é usado.

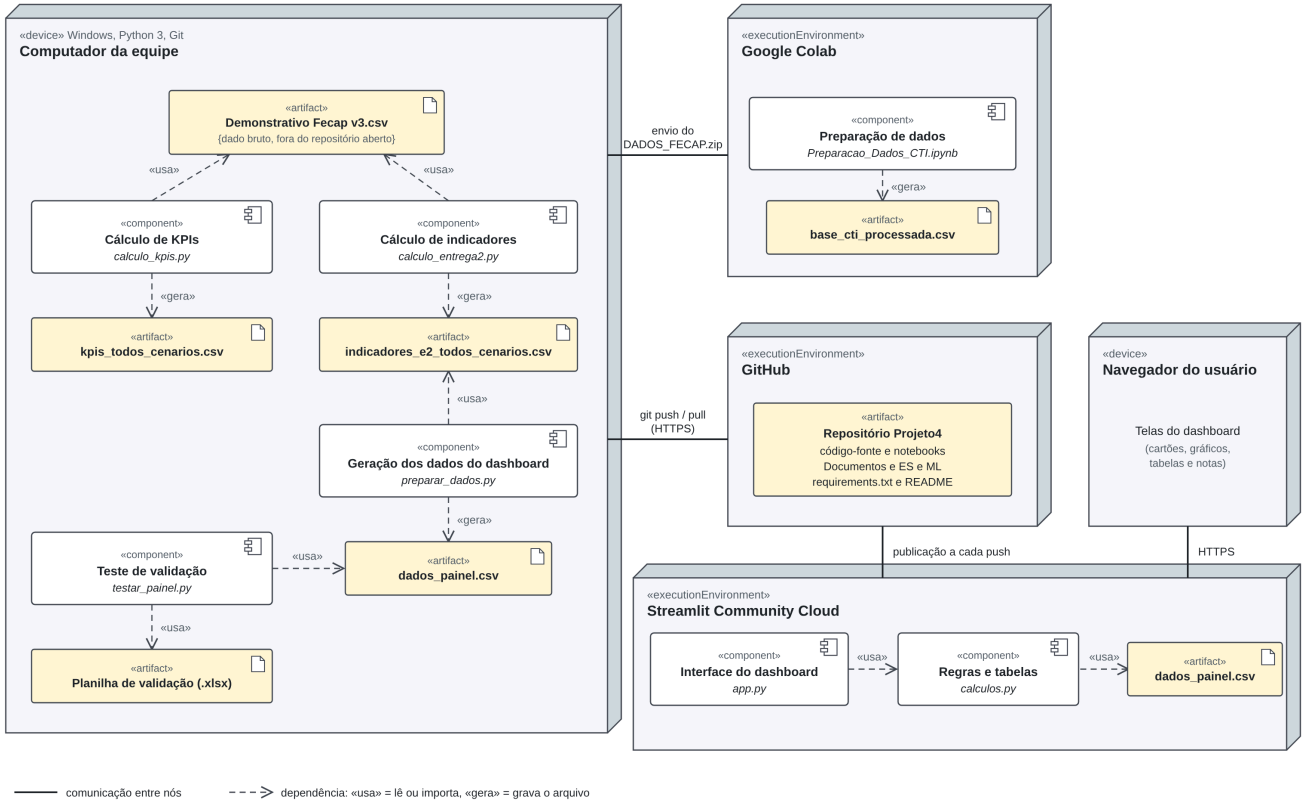


Figura 2 – Diagrama de Implantação e Componentes da plataforma.

Fluxo do Sistema

O fluxo segue sempre a mesma ordem: o arquivo da CTI entra, passa pelos scripts de preparação e cálculo e sai como bases processadas, testes conferidos e dashboard.

Entradas

- Arquivo Demonstrativo Fecap v3.csv, enviado pela CTI dentro do DADOS_FECAP.zip. Ele tem quatro colunas (ano, cenário, conta e valor), separadas por ponto e vírgula, com codificação latin1 e números no formato brasileiro.
- Contas de três demonstrativos, balanço (BAL), resultado (DRE) e fluxo de caixa (FLU), para 12 anos e 1.200 cenários.
- Parâmetros fixos definidos pela equipe, como o ano comercial de 360 dias e a unidade em R$ milhões.
- Planilha de validação, usada como referência nos testes.
- Filtros escolhidos pelo usuário no dashboard: cenário, ano e se quer ver a mediana dos cenários.

Processamento

1. Leitura: o arquivo é lido com a codificação e o separador certos, e as linhas sem conta são descartadas.

2. Padronização: os espaços duplicados nos nomes das contas são removidos, os valores viram número e são convertidos para R$ milhões, e os textos "Ano 1" e "Total Cen_1" viram números.
3. Organização: os dados saem do formato longo, com uma linha por conta, e viram uma tabela com uma linha por cenário e ano e uma coluna por conta.
4. Cálculo: os scripts calculam os indicadores para as 14.400 combinações de cenário e ano e gravam os resultados em CSV.
5. Conferência: os scripts checam se os demonstrativos fecham entre si, com diferença zero, e o testar_painel.py compara os resultados com a planilha de validação.
6. Geração dos dados do dashboard: o preparar_dados.py separa só as colunas usadas nas telas e grava o dados_painel.csv.
7. Apresentação: o dashboard lê o arquivo uma vez, guarda em cache, filtra pelo cenário e ano escolhidos e monta as telas. Se o arquivo não existir, ele para e mostra uma mensagem de erro.

Saídas

- Bases processadas em CSV: base_cti_processada.csv, kpis_todos_cenarios.csv, indicadores_e2_todos_cenarios.csv e dados_painel.csv.
- Resultado dos testes no terminal, com o número de comparações iguais e diferentes.
- Dashboard web com cartões, gráficos, tabelas por ano e notas metodológicas.
- Tabelas exportadas em CSV pelo próprio usuário.
- Relatórios em PDF e planilhas guardados na pasta Documentos do repositório.

Observações Reflexivas

Fazer o design e os diagramas antes de fechar o código ajudou o grupo a ver o sistema inteiro, e não só cada script separado. O diagrama de casos de uso deixou claro que quem usa o dashboard não somos nós, e sim o gestor da CTI e o professor. Isso mudou a forma como pensamos as telas, as mensagens de erro e o README.

Na prática, o projeto não saiu exatamente como foi planejado. A primeira divergência foi no cálculo. Na primeira versão do dashboard, parte das contas era refeita dentro do próprio app, e isso criava o risco de o dashboard e a planilha mostrarem números diferentes. Na versão atual, o dashboard só lê o resultado dos scripts de cálculo, e o teste automático confirma que os valores batem.

A segunda mudança veio dos requisitos. Depois da orientação do professor, o conteúdo de uma parte do dashboard mudou e tivemos que refazer o módulo. Como as camadas estavam separadas, só mexemos na geração dos dados do dashboard e nas telas. A leitura

do arquivo e os cálculos continuaram iguais, e foi aí que a separação em camadas mostrou o seu valor.

Também apareceram problemas que o planejamento não previa. O arquivo da CTI usa codificação latin1, alguns nomes de contas têm espaços duplicados, existem contas com nomes parecidos e o último ano não tem o balanço completo. Tivemos que tratar cada um desses casos no código, e no dashboard o dado que falta aparece como "n/a" com uma explicação.

No uso real também surgiram falhas de usabilidade e de instalação. No Windows, o comando streamlit não era reconhecido, então mudamos o README para usar python -m streamlit run app.py e fixamos as versões das bibliotecas no requirements.txt. No modo escuro, uma tabela com cores fixas ficou ilegível e alguns cartões cortavam os valores. Tiramos as cores fixas e diminuímos a quantidade de cartões por linha.

A maior diferença entre o UML e a realidade está na implantação. O diagrama prevê o dashboard no Streamlit Community Cloud, mas até esta entrega ele rodou no computador da equipe. O motivo é que os dados da CTI são confidenciais e um app público deixaria o arquivo exposto.

Para a entrega final, as melhorias que precisamos fazer são:

- Publicar o dashboard na nuvem a partir de um repositório privado, ou com uma amostra dos dados autorizada pela CTI.
- Juntar os scripts em um comando só, para rodar o fluxo de ponta a ponta sem depender da ordem manual.
- Organizar o repositório em pastas que sigam as camadas do design: dados brutos, dados processados, código e documentação.
- Transformar o testar_painel.py em testes com pytest e rodar no GitHub Actions a cada push.
- Completar a validação da estrutura do arquivo (colunas, anos e cenários esperados), que hoje é parcial, e mostrar uma mensagem clara quando vier algo diferente.
- Trocar o CSV de cerca de 10 MB por Parquet, para o dashboard carregar mais rápido.
- Atualizar os diagramas UML com a versão final do código e registrar as decisões de design nas issues do GitHub.

No geral, o design e o UML deixaram de ser só documentação e viraram uma forma de testar as ideias antes de programar. Quando algo mudou no meio do caminho, foi o design em camadas que evitou retrabalho maior.